
AUDITORIA COMPLETA DE SEGURANCA

Integracao Mercado Pago

Sistema: Espelho de Vendas - SaaS

Data: 14/08/2026

Classificacao: CONFIDENCIAL

Escopo da Auditoria:

- Seguranca de credenciais e autenticacao
- Protecao contra manipulacao de valores
- Idempotencia e controle de concorrancia
- Validacao de webhooks e protecao contra replay
- Integridade do ciclo de vida de pagamentos
 - Confiabilidade e tratamento de falhas
 - Performance e escalabilidade
 - Observabilidade e logs
- Conformidade OWASP Top 10 e ASVS

Sumario

1. Resumo Executivo
2. Nota Geral
3. Vulnerabilidades Criticas
4. Vulnerabilidades Altas
5. Vulnerabilidades Medias
6. Vulnerabilidades Baixas
7. Problemas de Performance
8. Problemas de Confiabilidade
9. Problemas de Arquitetura
10. Fluxo Atual
11. Fluxo Recomendado
12. Plano de Correcao
13. Correcoes Implementadas
14. Declaracao Final

1. Resumo Executivo

Esta auditoria avalia a integração do sistema Espelho de Vendas com o Mercado Pago para processamento de assinaturas recorrentes. A análise cobre toda a cadeia: frontend, backend (API routes Next.js), webhook receiver, banco de dados (Supabase/PostgreSQL), biblioteca de integração (mercadopago.ts) e schema de migração. O sistema utiliza o SDK oficial mercadopago v3.4.0 com clientes de PreApproval, Payment e PreApprovalPlan, além de verificação HMAC-SHA256 em webhooks.

Foram identificados 2 vulnerabilidades criticas, 5 vulnerabilidades altas, 7 vulnerabilidades medias e 5 vulnerabilidades baixas/informativas. Os problemas criticos incluem a ausencia de idempotencia na criacao de assinaturas (risco de duplicacao financeira) e a possibilidade de bypass de verificacao de webhook em modo de desenvolvimento. Existem tambem lacunas significativas em maquina de estados de pagamento, validacao de precos, protecao contra condicoes de corrida e observabilidade.

Veredito: NAO APROVADO PARA PRODUCAO. O sistema possui bloqueadores que devem ser corrigidos antes de processar pagamentos reais. Os principais riscos sao: duplicacao de assinaturas por concorrência, webhook sem idempotencia, ausencia de maquina de estados para transicoes de status, e falta de validacao de valor cobrado no servidor contra o valor do plano.

2. Nota Geral

[illegible]

Metodologia de pontuacao: Cada dimensao e avaliada de 0 a 10, considerando cobertura de requisitos de seguranca, resistencia a ataques, robustez de implementacao e adequacao a boas praticas de mercado. Notas abaixo de 5.0 indicam deficiencias significativas. A nota de prontidao para producao e ponderada pela existencia de bloqueadores criticos.

3. Vulnerabilidades Criticas

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
CRITICA	Ausencia de idempotencia na criacao de assinatura - Race Condition	src/app/api/subscriptions/create/route.ts linhas 48-105	Dois requests simultaneos do mesmo usuario superam a verificacao de assinatura ativa (linhas 49-61) e criam duas assinaturas no MP e no banco. Resultado: usuario cobrado duas vezes com acesso concedido duas vezes. O gap entre SELECT e INSERT nao e protegido por lock ou constraint UNIQUE.	Adicionar constraint UNIQUE parcial: CREATE UNIQUE INDEX idx_one_active_sub ON assinaturas(user_id) WHERE status IN ('active', 'pending'). Usar SELECT FOR UPDATE ou operacao atomica no check-then-insert.	P0
CRITICA	Webhook sem verificacao de assinatura em modo de desenvolvimento aceita payload forjado	src/app/api/webhooks/mercadopago/route.ts linhas 30-35	A condicao "const isDev = !process.env.MERCADOPAGO_WEBHOOK_SECRET" permite que QUALQUER request POST sem assinatura seja processado se a variavel nao estiver configurada. Se o ambiente de producao acidentalmente nao tiver a variavel configurada, um atacante pode enviar webhooks forjados marcando pagamentos como aprovados e concedendo acesso indevido.	Remover completamente o bypass isDev. Se o secret nao estiver configurado, o endpoint deve retornar 503 (Service Unavailable) em vez de processar o evento. Nunca desabilitar verificacao de assinatura.	P0

Atencao: As vulnerabilidades criticas representam riscos financeiros diretos. A duplicacao de assinatura (VULN-01) pode causar cobranca duplicada em cartao de credito do cliente, gerando chargebacks e perda de confianca. O bypass de webhook (VULN-02) permite concessao fraudulenta de acesso ao sistema. Ambas devem ser corrigidas antes de qualquer processamento de pagamento real.

4. Vulnerabilidades Altas

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
ALTA	Nenhuma validacao do valor cobrado contra o plano no webhook	src/app/api/webhooks/mercado_pago/route.ts linhas 134-161	O webhook aceita o valor "transaction_amount" do payload do Mercado Pago sem verificar se corresponde ao preco do plano. Embora o MP defina o valor, um comprometimento da conta MP ou um plano mal configurado poderia resultar em valor divergente. A assinatura e ativada independentemente do valor pago.	Apos upsert do pagamento, comparar paymentData.transaction_amount com o preco do plano vinculado a assinatura. Se divergir acima de uma tolerancia (ex: 5%), logar alerta e nao ativar automaticamente. Requerer confirmacao manual do admin.	P1
ALTA	Sem maquina de estados - transicoes arbitrariedades de status	src/app/api/webhooks/mercado_pago/route.ts linhas 226-261, admin/assinaturas/route.ts linhas 80-103	Nao existe validacao de transicoes validas de status. Um webhook ou admin pode fazer transicoes como rejected-confirmed active, cancelled-active, ou active-rejected sem nenhuma verificacao. O statusMap no webhook (linha 226) apenas mapeia, mas nao impede transicoes invalidas. O admin pode definir qualquer status valido sem checar o estado anterior.	Implementar maquina de estados com transicoes permitidas. Ex: pending-pending approved rejected cancelled expired; approved-cancelled expired paused; cancelled - sem transicoes de saida. Verificar status atual antes de atualizar.	P1
ALTA	Webhook consulta API do MP mas nao armazena idempotencia key	src/app/api/webhooks/mercado_pago/route.ts	Cada webhook recebido faz GET na API do MP (getMpPayment ou getMpSubscription) mesmo para eventos ja processados. Sem tabela de eventos processados ou mecanismo de idempotencia, webhooks duplicados (comuns no MP) geram chamadas repetidas a API externa e operacoes de banco redundantes. O upsert de pagamento mitiga parcialmente, mas a atualizacao de assinatura nao e idempotente.	Criar tabela webhook_events (id, event_id, type, processed_at, UNIQUE constraint em event_id). Antes de processar, verificar se ja existe. Usar a coluna como idempotency key. Se ja processado, retornar 200 imediatamente.	P1

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
ALTA	Payer email armazenado nos detalhes do pagamento (detalhes JSON)	src/app/api/webhooks/mercado_pago/route.ts linhas 147-153	O campo payer.email do Mercado Pago e salvo no campo "detalhes" (JSONB) da tabela pagamentos. Este dado pessoal pode ser acessado por qualquer admin e permanece sem prazo de expiracao ou criterio de retencao. Nao ha anonimizacao. Pode violar o principio de minimizacao de dados da LGPD.	Remover payer.email dos dados armazenados. Se necessario para conciliacao, armazenar apenas hash SHA-256 do email. Implementar politica de retencao e exclusao automatica apos periodo definido.	P1
ALTA	Admin pode definir status "active" sem verificar pagamento real	src/app/api/admin-sistema/assinaturas/route.ts linhas 91-93	O endpoint PATCH de assinaturas permite ao admin definir status=active sem nenhuma verificacao de pagamento existente, valor ou correspondencia com o MP. Um admin comprometido (ou com boas intencoes mas sem cuidado) pode conceder acesso a qualquer usuario sem pagamento, gerando prejuizo financeiro.	Quando status=active for definido pelo admin, exigir campo justificativa obrigatorio, registrar log de auditoria separado com user_id do admin, timestamp e IP. Considerar requerer confirmacao dupla ou restricao adicional para esta transicao.	P1

5. Vulnerabilidades Medias

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
MEDIA	Precos usando tipo numeric(10,2) mas logica JS usa Number (float)	mercadopago.ts linha 78, create/route.ts linha 82, webhook route.ts linha 178	Multiplos pontos do codigo convertem preco com Number() e realizam calculos com floating point. Na linha 178 do webhook, data_fim e calculada como "meses * 30 * 24 * 60 * 60 * 1000" - calculo impreciso que acumula erros. Para 12 meses, o resultado e 31.536.000.000 ms em vez dos dias reais. Nao causa exploracao direta, mas pode gerar discrepancia de datas.	Usar biblioteca de datas (date-fns, dayjs) para calculo de data_fim com meses calendario reais. Para precos, usar parseInt(valor * 100) / 100 para normalizar antes de persistir. Considerar usar centavos (integer) internamente.	P2
MEDIA	Tentativa de cancelamento no MP continua mesmo se falha, sem garantir consistencia	src/app/api/subscriptions/cancel/route.ts linhas 36-44	Se cancelar no MP falhar (linhas 37-43), o codigo faz "continue" e cancela localmente. O resultado e uma assinatura "cancelled_by_user" no banco mas ainda "active" no Mercado Pago, que continuara cobrando o usuario. Nao ha retry, nao ha reconciliacao, e o usuario nao e informado da falha parcial.	Implementar: (1) Se cancelamento no MP falhar, NAO cancelar localmente. Retornar erro para o usuario tentar novamente. (2) Adicionar job de reconciliacao periodico que compara status local vs MP. (3) Registrar falha para retry automatico.	P2
MEDIA	Ausencia de rate limiting em endpoints de pagamento	src/app/api/subscriptions/create/route.ts, /cancel/route.ts	Nenhum dos endpoints de pagamento possui rate limiting. Um atacante ou bug no frontend pode disparar centenas de criacoes de assinatura ou cancelamentos em sequencia. Cada criacao gera uma chamada a API do MP (custo, quota). Cancelamentos em massa podem causar indisponibilidade.	Implementar rate limiting por usuario (ex: 5 criacoes/hora, 3 cancelamentos/hora). Usar middleware ou decorator com store em Redis/Memory. Retornar 429 quando excedido.	P2

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
MEDIA	Sensibilidade de payer_id do webhook pode vaziar para usuarios	src/app/api/subscriptions/status/route.ts	O endpoint GET /api/subscriptions/status retorna dados da assinatura que podem incluir campos sensivos dependendo da evolucao do schema. A assinatura possui mercadopago_payer_id que, se exposto, pode ser usado para cross-reference com dados do MP. Embora atualmente o select nao inclua este campo, qualquer alteracao futura no select poderia expor dados.	Criar view SQL que exclui campos sensiveis (mercadopago_payer_id, mercadopago_subscription_id) das consultas de usuario. Usar SELECT explicito em vez de select(*). Aplicar denylist de campos em nivel de API.	P2
MEDIA	Funcao requireAdminSistema usa email hardcoded como fallback	src/lib/admin-auth.ts linha 4, 33	O email "prosperosdirecional@gmail.com" esta hardcoded como fallback de autorizacao. Se este email for comprometido ou se o perfil do usuario nao tiver role=admin_sistema por um bug, o acesso admin e concedido via email. Isso cria um vetor de ataque single-point-of-failure e dificulta rotacao de credenciais admin.	Remover o fallback por email. A autorizacao deve ser exclusivamente via role no perfil. Se o perfil nao existir ou nao tiver a role correta, negar acesso. Adicionar migration para garantir que o admin tenha a role configurada.	P2
MEDIA	Logs contem IDs de pagamento e subscription IDs	webhook route.ts linhas 46, 84, 126, 194, 264, 267	Multiplos console.log registram paymentId, subscriptionId e detalhes de eventos. Embora nao registrem o access token diretamente, a combinacao de IDs de pagamento com dados de ambiente pode ser util para um atacante com acesso aos logs. Em caso de vazamento de logs, os IDs podem ser usados para phishing ou social engineering.	Implementar log estruturado com niveis apropriados. Usar logger com sanitizacao que mascara IDs sensiveis em ambientes nao-debug. Remover console.log de producao ou substituir por logger configuravel.	P2

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
MEDIA	Sem validacao de input para campos de texto nos planos (admin)	src/app/api/admin-sistema/planos/route.ts linhas 92-101	Os campos nome e descricao aceitam qualquer string sem validacao de comprimento maximo ou conteudo. O campo features aceita qualquer array JSON sem limitar tamanho ou conteudo dos itens. Um admin pode criar planos com nomes HTML/script, JSON malformado, ou strings extremamente longas que poderiam causar problemas de renderizacao ou armazenamento.	Adicionar validacao: nome max 100 chars, descricao max 500 chars, features max 20 itens cada com max 200 chars. Sanitizar strings para evitar XSS se renderizadas no frontend. Validar que features e array de strings.	P2

6. Vulnerabilidades Baixas / Informativas

Severidade	Problema	Localizacao	Impacto / Exploracao	Correcao	Prioridade
BAIXA	CSS class usando expressao ternaria no PlanosClient que pode causar classname vazio	PlanosClient.tsx linha 306	Baixo impacto. Classe gerada dinamicamente com template literal que nao causa vulnerabilidade de seguranca, mas pode gerar inconsistencia visual se as condicoes nao forem mutuamente exclusivas.	Usar clsx ou cn() para gerenciar classes condicionais de forma mais segura.	P3
BAIXA	DELETE de plano usa JSON body em vez de path param	admin-sistema/planos/route.ts linhas 271-321	O metodo DELETE recebe o ID no body JSON, o que e incomum para DELETE e pode causar problemas com proxies/CDNs que fazem strip de body em DELETE. Nao e exploravel mas vai contra convencoes REST.	Migrar para DELETE /api/admin-sistema/planos/[id] com path param. Ou aceitar como decisao de design documentada.	P3
BAIXA	Erro generico "Erro interno" nao diferencia falhas para o cliente	Multiplos arquivos de rota	Todos os endpoints retornam mensagens genericas de erro em caso de falha. Embora seja bom para seguranca (nao vaza detalhes), dificulta o suporte ao cliente e a diagnostico de problemas. O admin precisa consultar logs para entender o erro.	Implementar codigos de erro semanticos (ex: PAYMENT_MP_UNAVAILABLE, PLAN_NOT_SYNCED) que o frontend pode traduzir para mensagens uteis sem expor detalhes tecnicos.	P3
BAIXA	Ausencia de paginacao na listagem de assinaturas (admin)	admin-sistema/assinaturas/route.ts linhas 9-46	A query retorna todas as assinaturas sem paginacao (.order sem .range). Com crescimento do numero de usuarios, esta query retornara datasets cada vez maiores, consumindo memoria e bandwidth desnecessariamente.	Implementar paginacao com cursor-based ou offset/limit. Adicionar parametros page e pageSize na query. Retornar metadados de paginacao (total, hasMore).	P3
INFORMATIVA	Singleton de clientes MP nao e thread-safe em cold start	mercadopago.ts linhas 15-57	As variaveis _client, _paymentClient e _planClient usam lazy init sem lock. Em ambientes serverless com cold starts simultaneos, e teoricamente possivel criar multiplas instancias. Na pratica, Next.js roda em single-thread, entao o risco e teorico.	Para robustez em ambientes serverless futuros, usar padrao de init condicional mais seguro ou inicializar no module scope. Nao e urgente.	P3

7. Problemas de Performance

Problema	Causa	Impacto	Solucao	Ganho Esperado
Webhook faz 2-3 queries sequenciais por evento	Busca assinatura por preapproval_id, depois por payment_id, depois busca plano para calculo de data_fim	Latencia adiciona 50-150ms por webhook. Em picos, pode causar timeout no MP que espera resposta rapida.	Combinar queries com JOIN. Usar query unica que busca assinatura + plano em uma operacao. Considerar cache do plano por 5 min.	Reducao de 40-60% de latencia por evento.
Admin busca todas assinaturas sem paginacao	Query sem .range() retorna todos os registros de uma vez	Com 1000+ assinaturas, response JSON pode ter 500KB+. Frontend congela renderizando tabela grande.	Implementar paginacao server-side com .range(page*size, (page+1)*size-1) e count total.	Response time constante independente do volume.
Singleton clients MP com timeout de 15s pode bloquear	Timeout de 15s (getMpConfig) e compartilhado entre todas as operacoes	Se API do MP ficar lenta, todas as requests de pagamento ficam presas por 15s. Connection pool pode esgotar.	Timeouts diferenciados: 5s para webhook (urgente), 10s para criacao, 15s para consultas admin.	Melhor responsividade em degradacao.
N+1 potencial na lista de assinaturas admin	Query JOIN com profiles e planos, mas sem otimizacao de select fields	Transferencia de dados desnecessaria. O JOIN com profiles traz campos que nao sao usados na listagem.	Selecionar apenas campos necessarios no JOIN. Remover campos nao utilizados do select.	Reducao de 20-30% do payload.

8. Problemas de Confiabilidade

8.1 Webhook - Idempotencia e Replay

O endpoint de webhook nao possui mecanismo de idempotencia. O Mercado Pago pode reenviar o mesmo webhook varias vezes (por timeout, retry ou ate mesmo comportamento normal da plataforma). Cada recebimento dispara: (1) chamada a API do MP para buscar detalhes, (2) upsert no banco, (3) atualizacao de status da assinatura. O upsert de pagamento usa `onConflict=mercadopago_payment_id` que e parcialmente idempotente, mas a atualizacao da assinatura (linhas 164-192) e executada toda vez sem verificacao se o status ja e o correto. Isto nao causa dano financeiro direto, mas desperdica recursos e pode gerar inconsistencia de timestamps (`data_inicio` atualizado multiplas vezes).

8.2 Concorrencia

A verificacao de assinatura ativa na criacao (`check-then-act`) e classica race condition. O fluxo e: SELECT assinatura ativa - se nao existe, INSERT nova assinatura. Entre o SELECT e o INSERT, outra request pode passar pela mesma verificacao. Sem lock no banco (`SELECT FOR UPDATE`) ou constraint UNIQUE parcial, as duas requests criam assinaturas duplicadas. O PostgreSQL suporta partial unique indexes que resolveriam isso de forma elegante e performatica.

8.3 Falhas de Rede

Se a API do MP retornar timeout apos criar a assinatura (linha 73-77 do `create/route.ts`), o usuario sera redirecionado para o checkout, mas o registro local pode nao ter sido criado (se a falha ocorrer nas linhas 85-104). Neste cenario, o usuario paga no MP mas o sistema nao tem registro da assinatura. O webhook de pagamento tentara encontrar a assinatura por `preapproval_id` (metadados do pagamento) e nao encontrara, logando apenas um warning. Nao ha mecanismo de recovery para este cenario. Alem disso, nao existem retries com backoff em nenhuma chamada a API do MP. Se uma chamada falhar por timeout transitivo, a operacao simplesmente falha sem tentar novamente.

8.4 Cancelamento Inconsistente

No cancelamento (`cancel/route.ts` linhas 36-44), se a chamada ao MP falhar, o sistema cancela localmente mesmo assim. O resultado e um estado inconsistente: local = cancelled, MP = active. O MP continuara cobrando. O usuario acha que cancelou mas sera cobrado novamente no proximo ciclo. Nao ha job de reconciliacao que detecte esta divergencia, e o usuario nao recebe nenhuma notificacao sobre a falha parcial do cancelamento.

8.5 Reconciliacao

Nao existe nenhum mecanismo de reconciliacao entre o estado local e o estado no Mercado Pago. Se um webhook for perdido (servidor fora do ar, falha de rede), o estado local ficara desatualizado permanentemente. E necessario implementar um job periodico (ex: a cada hora) que: (1) busque todas assinaturas ativas locais, (2) consulte o status no MP para cada uma, (3) atualize se divergir. Este job tambem deve detectar pagamentos aprovados no MP que nao foram registrados localmente.

9. Problemas de Arquitetura

9.1 Processamento Sincrono de Webhook. O webhook faz chamada a API do MP de forma sincrona dentro do handler HTTP. Se o MP estiver lento, o webhook demora para responder, e o Mercado Pago pode reenviar o evento (pensando que falhou). Idealmente, o webhook deveria: (1) validar assinatura, (2) persistir o evento bruto em uma fila, (3) responder 200 imediatamente, (4) processar o evento de forma assincrona. Isto reduz o risco de reenvios e melhora a resiliencia do sistema.

9.2 Acoplamento Forte entre Webhook e Ativacao. O webhook ativa a assinatura diretamente sem passar por uma camada de servico. Se a logica de ativacao precisar mudar (ex: adicionar grace period, trial, notificacao), sera necessario modificar o webhook. Uma camada de servico (SubscriptionService) centralizaria a logica e seria reutilizada pelo webhook, pelo admin e por jobs de reconciliacao.

9.3 Ausencia de Event Store. Nao ha registro imutavel de eventos de pagamento. A tabela pagamentos registra o estado atual, mas nao o historico de transicoes. Se um pagamento mudar de pending para approved e depois para refunded, so o estado final e preservado. Para auditoria financeira, e necessario ter o historico completo de transicoes de status com timestamp e origem (webhook, admin, usuario).

9.4 Configuracao de Planos Fragil. A alteracao de preco ou periodo de um plano ja sincronizado com o MP limpa o mercadopago_plan_id, exigindo re-sincronizacao manual. Se o admin alterar o preco e esquecer de re-sincronizar, o plano fica indisponivel para compra ate que a sincronizacao seja feita. Nao ha validacao no frontend que avise de forma mais explicita, nem mecanismo automatico de re-sincronizacao.

9.5 Falta de Separacao entre Ambientes. O codigo usa a mesma logica para sandbox e producao, sem indicador claro de ambiente. A unica diferenciacao e a presenca ou nao da variavel MERCADOPAGO_WEBHOOK_SECRET. Deveria haver validacao explicita de ambiente com bloqueios de operacoes perigosas em sandbox (ex: nao permitir criacao de assinatura real com token de sandbox configurado erroneamente).

10. Fluxo Atual

Passo	Descricao
1	Usuario acessa /planos e seleciona um plano
2	Frontend envia POST /api/subscriptions/create com { planoId }
3	Backend verifica autenticacao (Supabase auth)
4	Backend busca plano no banco (RLS: ativo=true)
5	Backend verifica se usuario ja tem assinatura ativa (SELECT sem lock)
6	Backend cria preapproval no Mercado Pago via SDK
7	Backend registra assinatura local com status pending
8	Backend retorna init_point (URL de checkout)
9	Frontend redireciona usuario para checkout do MP
10	Usuario realiza pagamento no checkout do MP
11	MP envia webhook POST /api/webhooks/mercadopago
12	Backend verifica assinatura HMAC-SHA256 do x-signature
13	Backend faz GET na API do MP para confirmar pagamento
14	Backend faz upsert na tabela pagamentos
15	Se aprovado, backend atualiza assinatura para active + calcula data_fim
16	Usuario acessa /assinatura e visualiza status

Pontos de confianca no fluxo atual: O passo 5 (verificacao de assinatura ativa) e o ponto mais critico - nao e atomico. O passo 13 (consulta ao MP) e positivo, mas o resultado nao e usado para validar o valor. O passo 15 (ativacao) nao verifica transicoes validas de estado nem compara valor pago vs. preco do plano.

11. Fluxo Recomendado

Passo	Descricao (com melhorias destacadas)
1	Usuario acessa /planos e seleciona um plano
2	Frontend envia POST /api/subscriptions/create com { planoId }
3	Backend verifica autenticacao e aplica rate limit (5 req/hora por usuario)
4	Backend busca plano no banco com locked row (SELECT FOR UPDATE)
5	Backend verifica assinatura ativa com constraint UNIQUE parcial (atomico)
6	Backend cria preapproval no MP com external_reference = userId:planoId
7	Backend registra assinatura local em transacao DB atomica
8	Backend retorna init_point
9	Frontend redireciona para checkout do MP
10	Usuario realiza pagamento
11	MP envia webhook - Backend verifica HMAC + timestamp
12	Backend verifica idempotencia (tabela webhook_events)
13	Backend faz GET no MP para confirmar status E valor
14	Backend compara valor pago vs. preco do plano (tolerancia 5%)
15	Backend registra pagamento + evento em transacao atomica
16	Se aprovado e valor OK, ativa assinatura com maquina de estados
17	Job de reconciliacao periodico (1h) compara local vs MP

12. Plano de Correcao

P0 - Bloqueador de Producao

Estes problemas devem ser corrigidos ANTES de processar qualquer pagamento real. Representam risco financeiro direto ou de seguranca critica.

- **VULN-01 (Idempotencia na criacao):** Adicionar partial unique index no PostgreSQL: `CREATE UNIQUE INDEX idx_one_active_sub ON assinaturas(user_id) WHERE status IN ('active', 'pending')`. Isto impede que duas assinaturas ativas/pendentes coexistam para o mesmo usuario, resolvendo a race condition a nivel de banco de dados.
- **VULN-02 (Bypass de webhook em dev):** Remover a variavel `isDev` e a logica de bypass. Se `MERCADOPAGO_WEBHOOK_SECRET` nao estiver configurado, retornar HTTP 503 com mensagem clara. Em ambiente de desenvolvimento, configurar um secret de teste. Nunca desabilitar a verificacao de assinatura.
- **VAL-01 (Validacao de valor no webhook):** Apos receber confirmacao do pagamento, buscar o preco do plano vinculado a assinatura e comparar com `transaction_amount`. Se a diferenca for maior que 5%, logar como alerta critico e nao ativar automaticamente. Enviar notificacao ao admin para revisao manual.

P1 - Critico

Corrigir imediatamente apos P0. Problemas que comprometem a integridade do sistema ou a seguranca da informacao.

- **Maquina de estados:** Implementar funcao de validacao de transicoes de status. Criar tabela ou tipo ENUM no PostgreSQL com transicoes validas. Aplicar tanto no webhook quanto no endpoint admin. Transicoes invalidas devem ser rejeitadas com HTTP 409.
- **Idempotencia de webhook:** Criar tabela `webhook_events` com UNIQUE em `(event_id)`. Antes de processar qualquer evento, verificar se ja foi processado. Retornar 200 imediatamente se sim. Isto elimina chamadas redundantes a API do MP.
- **Remocao de dados pessoais:** Remover `payer.email` do campo detalhes. Se necessario para conciliacao, armazenar hash. Implementar politica de retencao LGPD.
- **Auditoria de alteracoes admin:** Quando admin alterar status de assinatura, registrar em tabela de auditoria: quem, quando, qual mudanca, IP. Tornar this auditable.
- **Cancelamento atomico:** Se cancelamento no MP falhar, nao cancelar localmente. Retornar erro para o usuario. Implementar retry com backoff (3 tentativas, 2s/5s/10s).

P2 - Importante

Corrigir antes de escalar o numero de usuarios. Problemas que se tornam mais graves com o crescimento.

- **Calculo de datas:** Substituir calculo de data_fim (meses*30*dias) por biblioteca de datas que lida com meses variados. Usar date-fns addMonths ou equivalente.
- **Rate limiting:** Implementar rate limiter por usuario nos endpoints de pagamento. Sugestao: 5 criacoes/hora, 3 cancelamentos/hora, 20 consultas status/hora.
- **Paginacao:** Adicionar paginacao em todas as listagens admin (assinaturas, pagamentos).
- **Validacao de inputs admin:** Adicionar validacao de comprimento e tipo em todos os campos de criacao/edicao de planos.
- **Remover email hardcoded:** Eliminar fallback por email em requireAdminSistema.
- **Logs estruturados:** Substituir console.log por logger com niveis e sanitizacao.
- **Protecao de campos sensiveis:** Criar view SQL que exclui campos sensiveis das consultas de usuario final.

P3 – Melhoria

Otimizacoes e melhorias futuras que elevam a maturidade do sistema mas nao sao bloqueadoras para lancamento inicial.

- **Webhook assincrono:** Implementar fila de processamento para webhooks. Responder 200 imediatamente, processar em background. Isto melhora a confiabilidade e reduz risco de reenvios.
- **Reconciliacao periodica:** Job que compara estado local vs MP a cada hora. Detecta e corrige divergencias automaticamente.
- **Event store:** Tabela de historico de transicoes de status para auditoria completa.
- **Camada de servico:** Extrair logica de SubscriptionService reutilizavel.
- **Metricas e alertas:** Implementar metricas (pagamentos criados, aprovados, rejeitados, webhooks recebidos, latencia) e alertas automaticos.
- **Testes de seguranca:** Criar suite de testes para autorizacao, manipulacao de valores, webhook duplicado e concorrencia.

13. Correcoes Implementadas

As correcoes abaixo foram implementadas diretamente no codigo-fonte como parte desta auditoria. Cada correcao e descrita com o problema, a alteracao realizada e o arquivo modificado.

13.1 Remocao do Bypass de Verificacao de Webhook (P0)

Arquivo: `src/app/api/webhooks/mercadopago/route.ts`. **Problema:** A variavel `isDev` permitia processar webhooks sem verificacao de assinatura quando `MERCADOPAGO_WEBHOOK_SECRET` nao estava configurado. **Correcao:** A logica de bypass foi completamente removida. Agora, se o secret nao estiver configurado, o endpoint retorna HTTP 503 (Service Unavailable) com mensagem clara indicando que a integracao nao esta configurada. Se a assinatura for invalida, retorna 401. Isto garante que em producao, nenhum webhook sera processado sem verificacao, mesmo que uma variavel de ambiente falte acidentalmente.

13.2 Melhoria na Validacao de Status no Webhook

Arquivo: `src/app/api/webhooks/mercadopago/route.ts`. **Problema:** O webhook aceitava qualquer status retornado pelo MP e o normalizava para um conjunto valido, sem verificar se a transicao fazia sentido. **Correcao:** Foi adicionada validacao de transicoes de status no handler de pagamentos. O webhook agora verifica o estado atual da assinatura no banco antes de atualizar e so permite transicoes validas (`pending-confirmed`, `active`, `active-confirmed`, `cancelled/paused`, etc.). Transicoes invalidas sao logadas como warning e ignoradas.

13.3 Validacao de Valor do Pagamento (P0)

Arquivo: `src/app/api/webhooks/mercadopago/route.ts`. **Problema:** O valor do pagamento nao era comparado com o preco do plano. **Correcao:** Apos o upsert do pagamento, o webhook agora busca o preco do plano vinculado a assinatura e compara com o `transaction_amount`. Se a diferenca for maior que 5%, o sistema loga um alerta critico e nao ativa a assinatura automaticamente, requerendo intervencao manual do admin.

13.4 Idempotencia do Webhook com Tabela de Eventos (P1)

Arquivo: `supabase/migration-subscriptions.sql`. **Problema:** Webhooks duplicados eram processados multiplas vezes. **Correcao:** Foi adicionada a tabela `webhook_events` com UNIQUE constraint em (`event_id`). O webhook verifica esta tabela antes de processar. Se

o evento ja foi processado, retorna 200 imediatamente sem re-processar.

13.5 Correcao do Cancelamento Atomico (P1)

Arquivo: src/app/api/subscriptions/cancel/route.ts. **Problema:** Cancelamento local era executado mesmo se o cancelamento no MP falhasse. **Correcao:** Se a chamada ao MP falhar, o sistema agora retorna erro ao usuario (502) sem cancelar localmente. O usuario pode tentar novamente. O erro e logado para monitoramento e futuro job de reconciliacao.

13.6 Constraint de Assinatura Unica por Usuario (P0)

Arquivo: supabase/migration-subscriptions.sql. **Problema:** Race condition na criacao de assinatura. **Correcao:** Adicionada instrucao para criar partial unique index: CREATE UNIQUE INDEX idx_one_active_sub ON assinaturas(user_id) WHERE status IN ('active', 'pending'). Esta constraint impide a nivel de banco que um usuario tenha mais de uma assinatura ativa ou pendente.

14. Declaracao Final

NAO APROVADO PARA PRODUCAO

Motivo: Existem vulnerabilidades criticas (P0) que foram identificadas e tiveram correcoes preparadas nesta auditoria, mas que dependem de: (1) execucao da migration SQL adicional no Supabase (webhook_events table + partial unique index), (2) configuracao das variaveis de ambiente em producao, e (3) validacao e teste das correcoes implementadas.

Bloqueadores restantes para producao:

- Executar migration adicional no Supabase (webhook_events + idx_one_active_sub)
- Configurar MERCADOPAGO_ACCESS_TOKEN com credenciais de PRODUCAO
- Configurar MERCADOPAGO_WEBHOOK_SECRET com secret de producao
- Configurar NEXT_PUBLIC_APP_URL com URL de producao
- Configurar webhook URL no dashboard do Mercado Pago
- Testar fluxo completo em sandbox antes de ir para producao
- Validar que as correcoes de seguranca nao quebram o fluxo legitimo
- Implementar reconciliacao periodica (P3, mas recomendado antes de producao)

Apos a execucao das migrations, configuracao das variaveis de ambiente e validacao dos testes, o sistema atingira um nivel adequado de seguranca para operacao inicial em producao. As melhorias P2 e P3 devem ser implementadas progressivamente conforme o sistema ganha usuarios e volume de transacoes.

Nota importante: Esta auditoria foi realizada com base na analise estatica do codigo-fonte. Testes dinamicos (penetration testing, load testing) nao foram executados e sao recomendados como etapa complementar antes do lancamento em producao. A analise considerou o comportamento esperado do SDK do Mercado Pago com base na documentacao oficial, mas nao inclui testes reais contra a API do Mercado Pago.